@qa_insights

KUBERNETES COMPLETE NOTES

(Kubernetes)

BEGINNER TO ADVANCED

SWIPE TO NEXT

FOLLOW | SHARE | COMMENT | LIKE

@qa_insights

1. Kubernetes

★ What is Kubernetes?

Kubernetes (K8s) is an open-source container orchestration platform used to automate the deployment, scaling, and management of containerized applications.

It groups containers that make up an application into logical units for easy management and discovery.

★ Why Kubernetes?

- To manage containers at scale.
- To automate deployment, scaling and operations.
- To ensure high availability and reliability.
- To optimize resource utilization.
- To provide portability across environments (cloud, on-prem, hybrid).

★ Key Features

① Automation – Automates deployment, scaling, and management.
② Scalability – Easily scale apps up or down based on demand.
③ Self-Healing – Automatically restarts, replaces or reschedules failed containers.
④ High Availability – Ensures minimal downtime with replication and failover.
⑤ Load Balancing – Distributes traffic across containers.
⑥ Service Discovery – Automatically discovers services and endpoints.
⑦ Rolling Updates & Rollbacks – Update apps without downtime and rollback easily if needed.
⑧ Portability – Run applications anywhere: on-prem, cloud or hybrid.

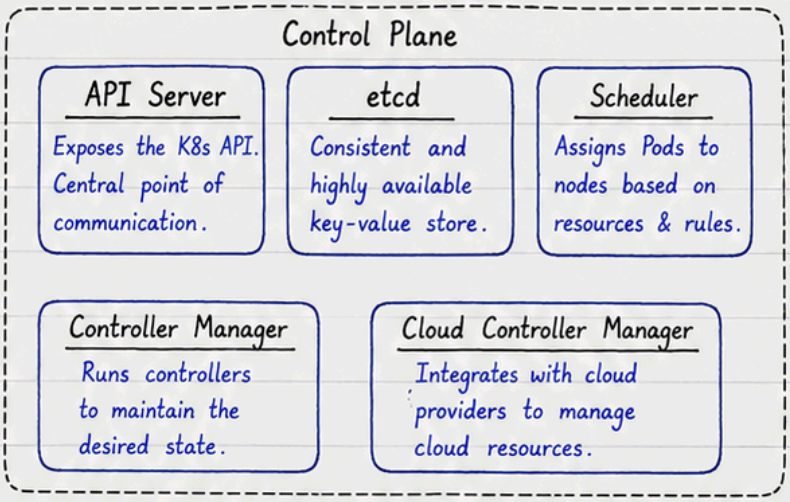
@qa_insights

2. Kubernetes Architecture

kubernetes

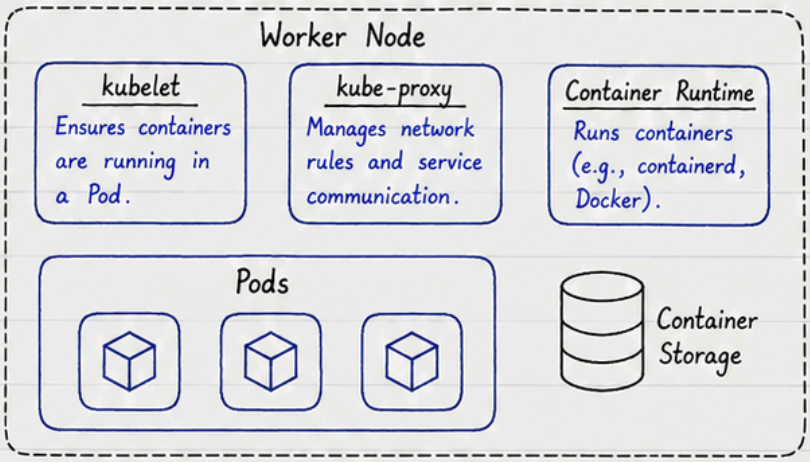
1) Control Plane (Master Node)

- Brain of the cluster.
- Manages the cluster and makes global decisions.
- Handles scheduling, scaling, and maintaining desired state.



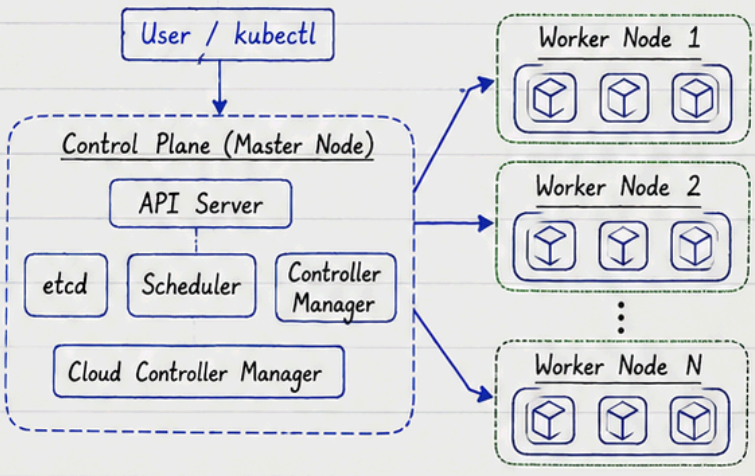
2) Worker Node

- Runs the actual applications (containers).
- Executes pods as instructed by the control plane.
- Contains kubelet, kube-proxy and container runtime.



3) Cluster Overview

- A Kubernetes cluster is a set of master (control plane) and worker nodes working together.
- Control plane manages the cluster.
- Worker nodes run the workloads (pods).



★ Control Plane thinks, Worker Nodes do the work.

@qa_insights

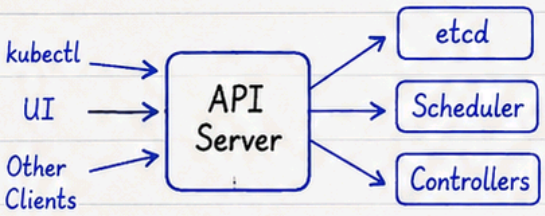
3. Core Components

kubernetes

These are the main building blocks that make Kubernetes work.

① API Server

- Frontend for the Kubernetes control plane.
- Exposes the Kubernetes API and is the central point of all communication.



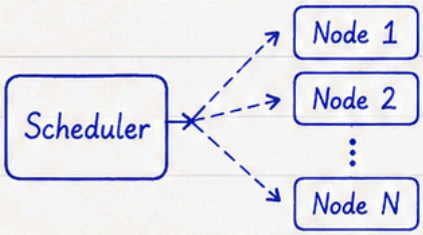
② etcd

- Consistent and highly available key-value store.
- Stores all cluster data and configuration in a reliable way.

Stores cluster state and configuration.

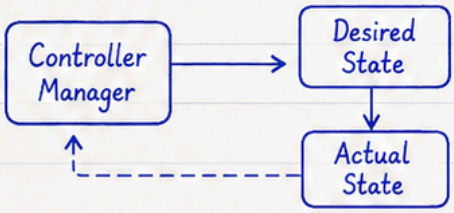
③ Scheduler

- Watches for newly created Pods with no node assigned.
- Selects the best node for the Pod to run on based on resources and policies.



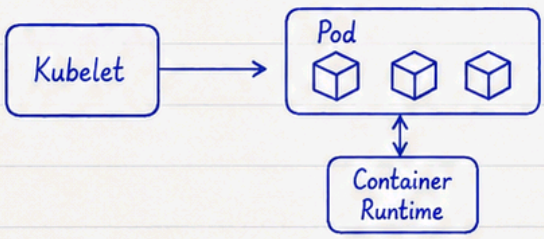
④ Controller Manager

- Runs controllers to maintain the desired state of the cluster.
- Examples: Node Controller, ReplicaSet Controller, Endpoint Controller, etc.



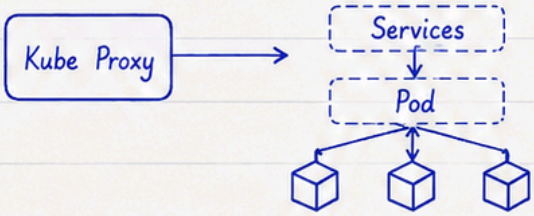
⑤ Kubelet

- Runs on each worker node.
- Ensures containers are running in a Pod.
- Communicates with the API Server.



⑥ Kube Proxy

- Runs on each node.
- Maintains network rules.
- Enables network communication to Pods from inside and outside the cluster.



★ Together, these components ensure the cluster is healthy, scalable, and self-healing.

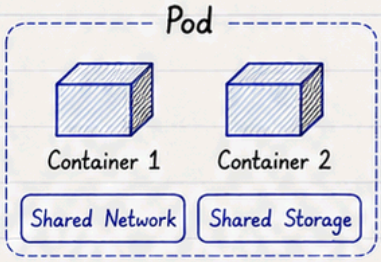
@qa_insights

4. Pods, ReplicaSet, Deployment, Rolling Update & Rollback

kubernetes

① Pods

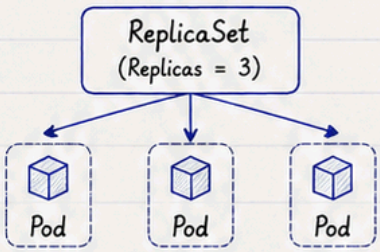
- The smallest deployable unit in Kubernetes.
- Represents one or more containers that share the same network, storage and configuration.
- Pods are ephemeral (can be created, destroyed and recreated).



Pods get their own IP and communicate with other Pods/Services.

② ReplicaSet

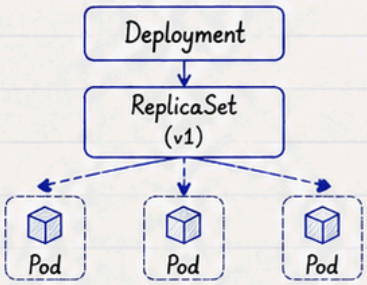
- Ensures that a specified number of Pod replicas are running at any given time.
- If a Pod fails, ReplicaSet creates a new Pod to maintain the desired count.



Maintains the desired number of identical Pods.

③ Deployment

- Manages ReplicaSets and provides declarative updates for Pods and ReplicaSets.
- Supports scaling, rolling updates and rollbacks.



Deployment manages ReplicaSets and ensures desired state of the application.

④ Rolling Update

- Updates Pods gradually with zero downtime.
- New Pods are created first, old Pods are terminated later.

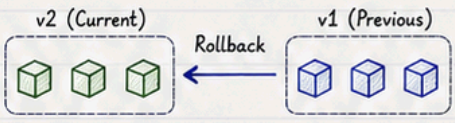
Old Pod (v1)

New Pod (v2)

Example (Replicas = 3)

⑤ Rollback

- Rolls back the Deployment to a previous version if something goes wrong.
- Restores the old ReplicaSet and Pods.



Quickly revert to a stable version.

★ Key Points

- Pod → smallest unit
- ReplicaSet → maintains desired pods
- Deployment → manages ReplicaSets, updates, scaling
- Rolling Update → zero downtime updates
- Rollback → revert to previous stable version

@qa_insights

5. Services

kubernetes

A Service in Kubernetes is a stable network endpoint that exposes a set of Pods as a network service.

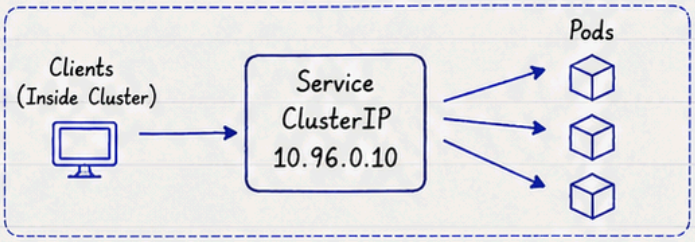
It provides load balancing, service discovery and a stable IP/DNS name.

Why Services?

- Stable endpoint for Pods
- Load balancing
- Service discovery
- Decouples frontend & backend

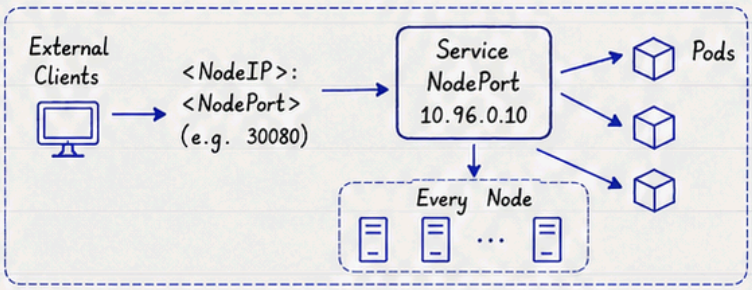
① ClusterIP (Default)

- Exposes the Service on a cluster-internal IP.
- Accessible only within the cluster.
- Default type if no type is specified.



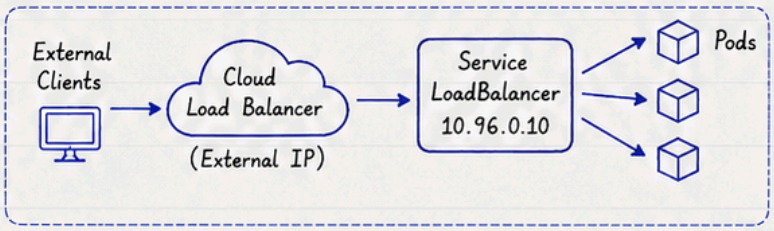
② NodePort

- Exposes the Service on a static port on each Node's IP.
- Accessible from outside the cluster using <NodeIP>:<NodePort>
- Port range: 30000 - 32767



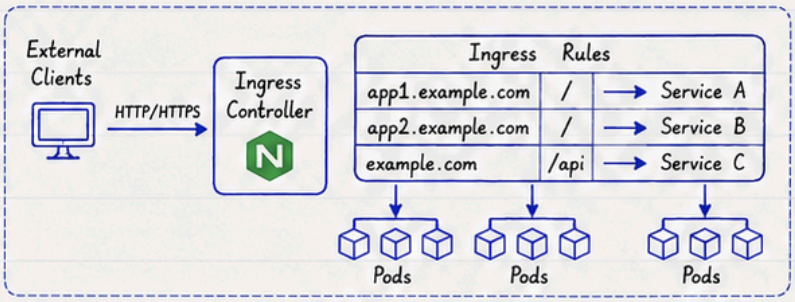
③ LoadBalancer

- Exposes the Service externally using a cloud provider's load balancer.
- Automatically provisions a load balancer (in supported clouds).
- Gets a stable external IP.



④ Ingress

- Provides HTTP/HTTPS routing to Services based on host names or paths.
- Uses an Ingress Controller (e.g. NGINX, Traefik) to manage the rules.
- Operates at Layer 7 (Application Layer).



★ Quick Comparison

ClusterIP	NodePort	LoadBalancer	Ingress
Internal access only. Default.	External access via <NodeIP>:<NodePort>. Static port on every node.	External access via cloud load balancer. Auto-provisioned.	HTTP/HTTPS based routing. Path/host based rules. Needs Ingress Controller.

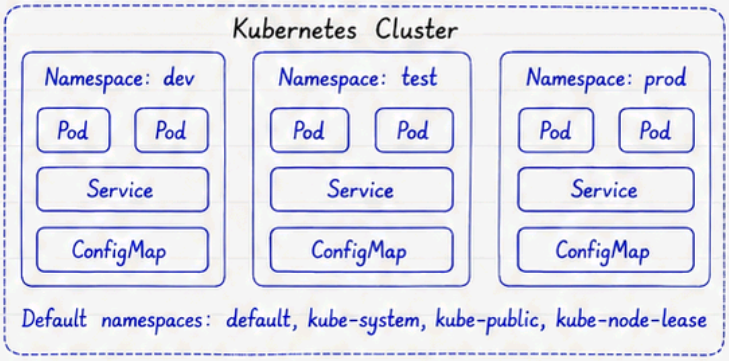
@qa_insights

6. Namespaces, Labels, Selectors, ConfigMap & Secret

kubernetes

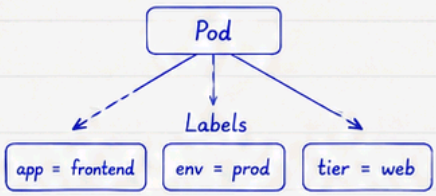
① Namespaces

- Namespaces provide a way to divide cluster resources between multiple users or projects.
- Helps in organizing and isolating resources within the same cluster.
- Resources in one namespace cannot access resources in another namespace (by default).



② Labels

- Labels are key-value pairs attached to Kubernetes objects.
- Used to organize and identify subsets of objects.
- Example: app=frontend, env=prod

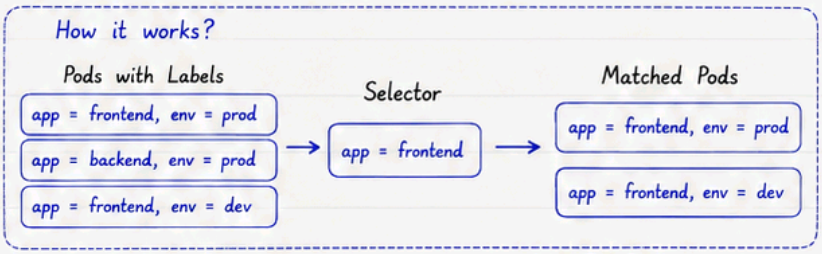


Example (YAML)

```yaml
metadata:
  name: my-pod
  labels:
    app: frontend
    env: prod
    tier: web
```

③ Selectors

- Selectors are used to filter resources based on labels.
- Used by Services, ReplicaSets, Deployments to find matching Pods.
- Types: Equality-based, Set-based

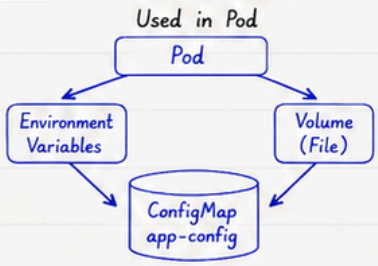


④ ConfigMap

- ConfigMap stores non-confidential configuration data in key-value pairs.
- Can be used as environment variables, files or command-line arguments.
- Helpful to separate config from code.

Example (YAML)

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  app.name: QA Insights
  app.mode: production
  log.level: info
```

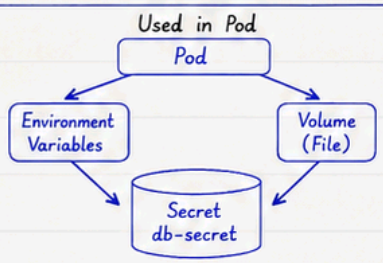


⑤ Secret

- Secret stores sensitive data like passwords, tokens, keys, etc.
- Data is stored in base64 encoded format (not encrypted by default).
- Used as environment variables or mounted as files in Pods.

Example (YAML)

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
data:
  username: YWRtaW4=
  password: cGFzc3dvcmQ=
    (base64 encoded)
```



★ Key Takeaway

- Namespaces → organize & isolate
- ConfigMap → non-sensitive config
- Labels → identify objects
- Secret → sensitive data
- Selectors → filter objects

@qa_insights

7. Volumes, PV, PVC & StorageClass

kubernetes

Kubernetes Volumes provide persistent storage for containers. PV, PVC and StorageClass work together to make storage dynamic, scalable and manageable.

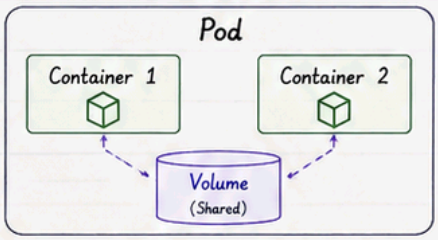
Why Volumes?

- ★ Data persists beyond Pod lifecycle
- ★ Decouples storage from Pod
- ★ Supports scalable & dynamic provisioning
- ★ Enables data sharing between containers

① Volumes

- Volumes are directories, possibly with some data in them, that are accessible to containers in a Pod.
- Types: emptyDir, hostPath, configMap, secret, persistentVolume, etc.
- Persistent data in Kubernetes is achieved using PV & PVC.

Pod with Volume



Volume Benefits

- ✓ Data available for the life of the Pod
- ✓ Shared between containers
- ✓ Manages storage lifecycle

② Persistent Volume (PV)

- PV is a piece of storage in the cluster provisioned by an admin or dynamically using StorageClass.
- It is a cluster resource, independent of any Pod.
- Has capacity, access modes and reclaim policy.

Persistent Volume (PV)

- Provisioned by Admin or dynamically
- Exists in the cluster
- Has capacity & access modes
- Reclaim Policy: Retain / Delete / Recycle

PV Example (YAML)

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: pv-demo
spec:
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: fast-storage
  hostPath:
    path: /data/pv-demo
```

③ Persistent Volume Claim (PVC)

- PVC is a request for storage by a user.
- It is namespace-scoped.
- Binds to a suitable PV that matches its requirements.
- Used by Pods to consume storage.

Persistent Volume Claim (PVC)

- Requested by user
- Namespace scoped
- Specifies size & access modes
- Binds to a matching PV

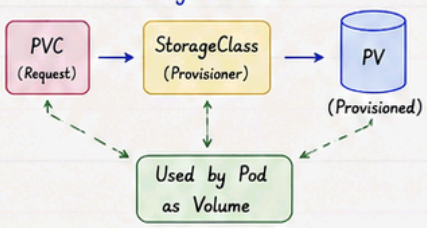
PVC Example (YAML)

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: pvc-demo
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 5Gi
  storageClassName: fast-storage
```

④ StorageClass

- StorageClass provides a way to describe "classes" of storage.
- Enables dynamic provisioning of PVs.
- Defines the provisioner, parameters and reclaim policy.
- PVC refers to a StorageClass to get dynamic storage.

StorageClass Flow



StorageClass Example (YAML)

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: fast-storage
provisioner: kubernetes.io/no-provisioner
parameters:
  type: hostpath
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
```

★ Key Takeaways

StorageClass defines how storage is provisioned → PVC requests storage → PV is provisioned (static or dynamic) → Pod uses the PVC to mount volume and store data

Remember:

- PV is cluster scoped (not namespace)
- PVC is namespace scoped
- Pod uses PVC, not PV directly
- StorageClass enables dynamic provisioning

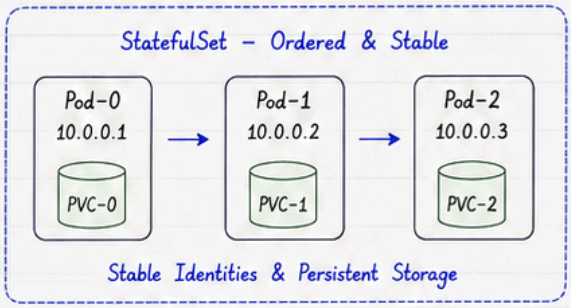
@qa_insights

8. StatefulSet, DaemonSet, Job & CronJob

kubernetes

① StatefulSet

- Manages stateful applications.
- Provides stable, unique network identities and persistent storage.
- Pods are created in order and deleted in reverse order.
- Examples: Databases (MySQL, MongoDB), Kafka, etc.

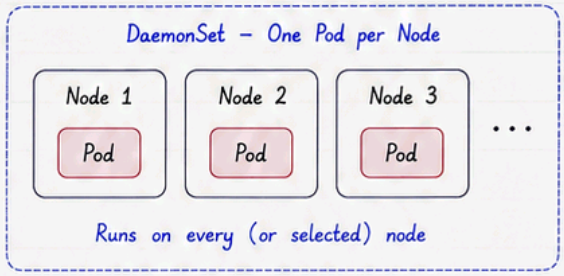


Example (YAML)

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql
spec:
  serviceName: mysql
  replicas: 3
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
```

② DaemonSet

- Ensures that a copy of a Pod runs on all (or selected) Nodes.
- Useful for node-level tasks like logging, monitoring, networking.
- When a new node is added, a Pod is automatically scheduled on it.

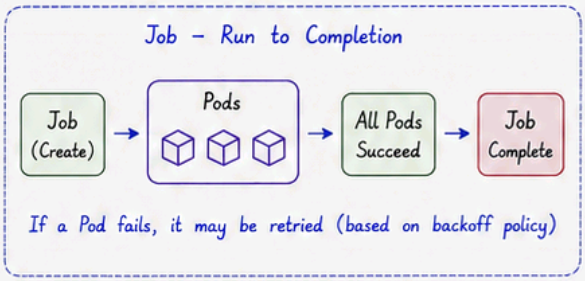


Example (YAML)

```yaml
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluentd
spec:
  selector:
    matchLabels:
      app: fluentd
  template:
    metadata:
      labels:
        app: fluentd
```

③ Job

- Creates Pods that run to completion and then stop.
- Suitable for one-time tasks like batch processing, data migration.
- Ensures that a specified number of Pods successfully complete.

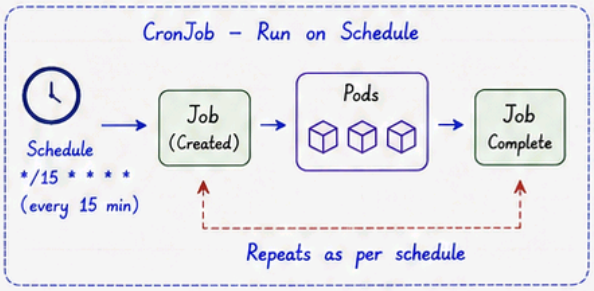


Example (YAML)

```yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: data-migration
spec:
  completions: 1
  template:
    spec:
      containers:
      - name: migrate
        image: my-app
      restartPolicy: OnFailure
```

④ CronJob

- Runs Jobs on a schedule (like cron).
- Useful for recurring tasks such as backups, reports, cleanup jobs.
- Creates a Job at the scheduled time.
- Example schedule format: "*/5 * * * *" (every 5 minutes)




```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: backup-job
spec:
 schedule: "0 2 * * *"
 jobTemplate:
  spec:
   template:
    spec:
     containers:
     - name: backup
       image: my-backup
     restartPolicy: OnFailure
```

★ Quick Comparison

Workload	Purpose	Key Points	Example Use Cases
StatefulSet	Stateful applications	Stable identity, ordered, persistent storage	Databases, Kafka, etc.
DaemonSet	Run on all nodes	One Pod per node	Logging agents, monitoring, CNI
Job	Run once to completion	Terminates after success/failure	Batch jobs, data migration
CronJob	Run on a schedule	Creates Jobs based on cron schedule	Backups, reports, cleanup

Think: Deployment = stateless | StatefulSet = stateful | Job/CronJob = tasks | DaemonSet = nodes

@qa_insights

9. Scheduling

kubernetes

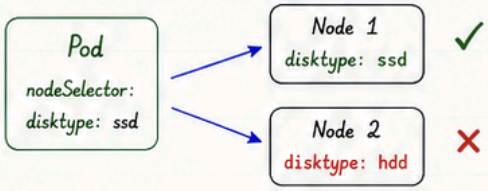
Scheduling is the process of assigning Pods to Nodes based on resource requirements, policies and constraints.

Scheduler Flow

① Node Selector

- Assigns Pods to Nodes with specific labels.
- Simple way to control placement.
- Uses exact match.

How it works



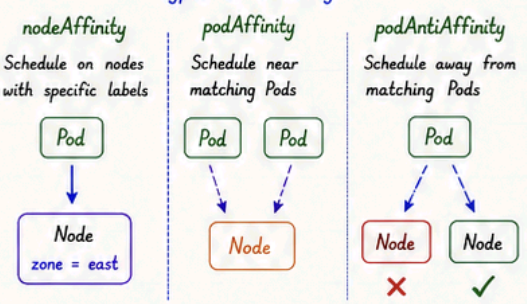
Example (YAML)

```yaml
spec:
  nodeSelector:
    disktype: ssd
    env: prod
```

② Affinity

- More advanced rules for scheduling.
- Types:
  - nodeAffinity (on nodes)
  - podAffinity (with other Pods)
  - podAntiAffinity (avoid Pods)
- Supports soft (preferred) & hard (required) rules.

Types of Affinity



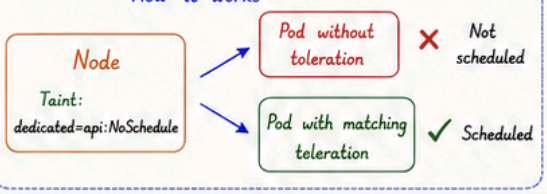
Example (YAML)

```yaml
affinity:
  nodeAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      nodeSelectorTerms:
      - matchExpressions:
        - key: disktype
          operator: In
          values: [ssd]
  podAntiAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
      labelSelector:
        matchLabels:
          app: web
      topologyKey: kubernetes.io/hostname
```

③ Taints

- Applied to Nodes to repel Pods.
- Pods without matching Toleration will not be scheduled.
- Helps reserve Nodes for specific workloads.

How it works



Example (YAML)

```yaml
# Taint on Node
kubectl taint nodes node1 dedicated=api:NoSchedule

# Pod without toleration → won't be scheduled
# Pod with toleration → will be scheduled
```

④ Tolerations

- Applied to Pods to allow them to be scheduled on tainted Nodes.
- Must match the taint key, value and effect.

How it works

Example (YAML)

```yaml
tolerations:
- key: "dedicated"
  operator: "Equal"
  value: "api"
  effect: "NoSchedule"
```

⑤ Taints & Tolerations Together

- Taints repel Pods.
- Tolerations allow Pods.
- Used together to control where Pods can run.

Summary

Common Effects

- NoSchedule → Pod won't be scheduled
- PreferNoSchedule → Try to avoid Node
- NoExecute → Evict existing Pods (if toleration not present)

★ Quick Comparison

Concept	Purpose	Applies To	Effect
Node Selector	Simple label based selection	Pod	Schedules Pod on matching labeled Nodes
Affinity	Advanced placement rules	Pod	Uses rules to place Pods near/away from others or nodes
Taints	Repel Pods from Nodes	Node	Pods without toleration won't be scheduled
Tolerations	Allow Pods on tainted Nodes	Pod	If toleration matches the taint

@qa_insights

10. Security

kubernetes

Kubernetes provides multiple security mechanisms to protect your cluster, applications and data.

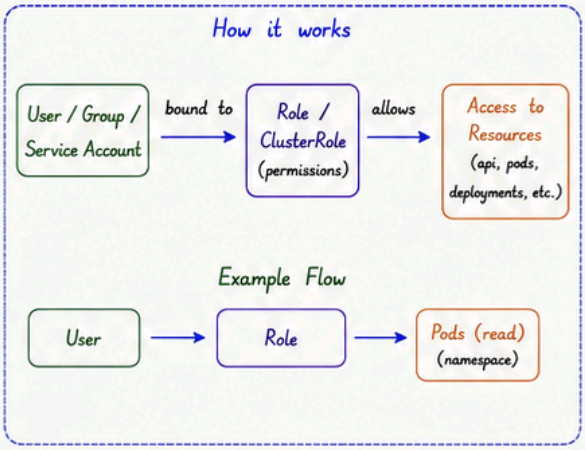
Security Goals

- ★ Authentication – Who are you?
- ★ Authorization – What can you do?
- ★ Isolation – Keep workloads separated
- ★ Audit & Monitoring – Track & detect issues

① RBAC

(Role Based Access Control)

- RBAC controls who can perform actions on Kubernetes resources.
- Use Roles (namespace scoped) and ClusterRoles (cluster scoped).
- Bind them to users, groups or service accounts.
- Principle of Least Privilege.

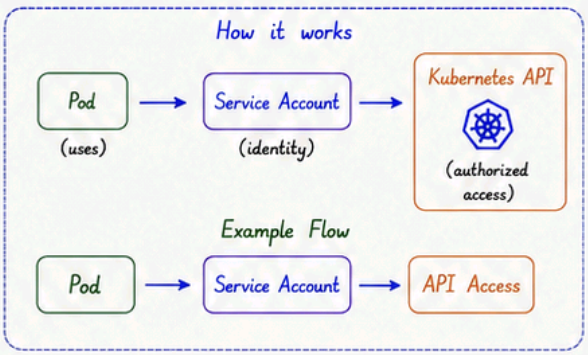


Example (YAML)

```yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: pod-reader
  namespace: dev
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
---
kind: RoleBinding
metadata:
  name: read-pods
  namespace: dev
subjects:
- kind: User
  name: alice
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

② Service Account

- Service Accounts provide an identity for processes running in Pods.
- Used by Pods to interact with the Kubernetes API.
- Each namespace has a default Service Account.
- Use least privilege permissions.

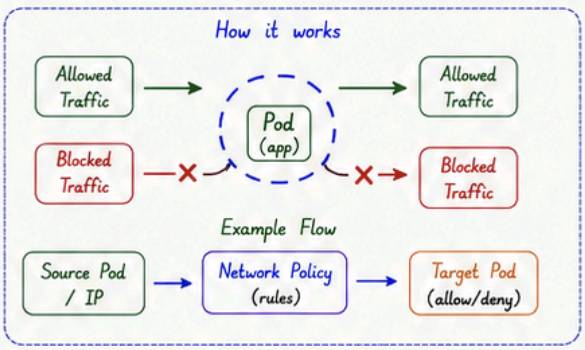


Example (YAML)

```yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: my-sa
  namespace: dev
---
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
  namespace: dev
spec:
  serviceAccountName: my-sa
  containers:
  - name: app
    inage: nginx
```

③ Network Policy

- Network Policies control the traffic flow to and from Pods.
- Define allowed/denied rules based on Pods, namespaces, IP blocks and ports.
- Provides isolation between applications.



Example (YAML)

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-frontend-to-backend
  namespace: app
spec:
  podSelector:
    matchLabels:
      app: backend
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          app: frontend
    ports:
    - protocol: TCP
      port: 80
```

★ Key Takeaways

- RBAC controls what you can do.
- Service Account is the identity for applications.
- Network Policies control how Pods communicate.
- Follow least privilege and defense in depth principles.

Best Practices

- Use RBAC – don't use cluster-admin.
- Create dedicated Service Accounts.
- Restrict network traffic with policies.
- Keep monitoring & auditing enabled.

@qa_insights

11. kubectl Cheat Sheet

Top 30-40 Most Used Commands

kubernetes

Tip: Use kubectl get <resource> -o wide for more details
Add -n <namespace> to run in a specific namespace

Abbreviations:
ns = namespace
svc = service
rs = replicaset
pod = pod
deploy = deployment
cm = configmap

#	Command	Purpose	#	Command	Purpose
1	kubectl version	Show client & server version	21	kubectl rollout history deploy/<deploy>	Show rollout history
2	kubectl cluster-info	Display cluster information	22	kubectl rollout undo deploy/<deploy>	Rollback to previous version
3	kubectl get nodes	List all nodes	23	kubectl get rs	List ReplicaSets
4	kubectl get nodes -o wide	List nodes with details	24	kubectl get rs -o wide	List ReplicaSets with details
5	kubectl get pods	List pods in current ns	25	kubectl get ns	List namespaces
6	kubectl get pods -A	List pods in all namespaces	26	kubectl get cm	List ConfigMaps
7	kubectl get pods -o wide	List pods with node/IP info	27	kubectl describe cm <cm>	Show ConfigMap details
8	kubectl describe pod <pod>	Show details of a pod	28	kubectl get secret	List Secrets
9	kubectl logs <pod>	Show logs of a pod	29	kubectl describe secret <secret>	Show Secret details
10	kubectl logs -f <pod>	Stream logs of a pod	30	kubectl get pv	List Persistent Volumes
11	kubectl exec -it <pod> --/bin/sh	Open shell in a pod	31	kubectl get pvc	List Persistent Volume Claims
12	kubectl get all	List all resources in ns	32	kubectl describe pvc <pvc>	Show PVC details
13	kubectl get all -A	List all resources in all ns	33	kubectl get ingress	List Ingress resources
14	kubectl get svc	List services	34	kubectl describe ingress <ing>	Show Ingress details
15	kubectl get svc -o wide	List services with details	35	kubectl top nodes	Show node resource usage
16	kubectl describe svc <svc>	Show details of a service	36	kubectl top pods	Show pod resource usage
17	kubectl get deploy	List deployments	37	kubectl apply -f <file.yaml>	Create/Update from YAML
18	kubectl get deploy -o wide	List deployments with details	38	kubectl delete -f <file.yaml>	Delete resources from YAML
19	kubectl describe deploy <deploy>	Show deployment details	39	kubectl delete pod <pod>	Delete a pod
20	kubectl rollout status deploy/<deploy>	Check rollout status	40	kubectl drain <node> --ignore-daemonsets	Drain a node for maintenance

Useful Flags

✓ -n <namespace>	Run command in a namespace
✓ -A	Run command in all namespaces
✓ -o wide	Show more details
✓ -o yaml	Output in YAML format
✓ -o json	Output in JSON format
✓ -w	Watch for changes
✓ -l <label>	Filter by label
✓ --all-namespaces	Same as -A

Common Resource Types

✓ pod	- Pod	✓ cm	- ConfigMap
✓ svc	- Service	✓ secret	- Secret
✓ deploy	- Deployment	✓ pv	- PersistentVolume
✓ rs	- ReplicaSet	✓ pvc	- PersistentVolumeClaim
✓ ns	- Namespace	✓ ing	- Ingress

Remember

- Everything in Kubernetes is API driven.
- Most kubectl commands follow the pattern: kubectl <command> <resource>
- Use describe & logs for troubleshooting.
- Use apply for idempotent changes.

Key Takeaways

- kubectl is your primary CLI tool to interact with the Kubernetes API.
- get shows the list, describe shows details.
- logs & exec help in debugging applications.
- rollout commands help in deployments & rollback.
- Security & monitoring commands keep your cluster healthy.

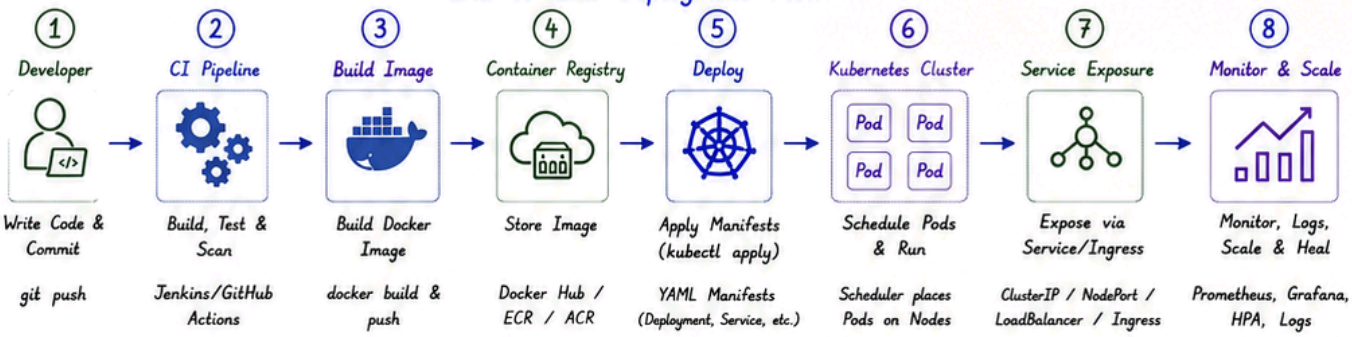
@qa_insights

12. Kubernetes Workflow

End-to-End Deployment Flow
From Code to Running Application

kubernetes

End-to-End Deployment Flow



Kubernetes Workflow Details

1. Developer	Developer writes code and pushes to Git repository.
2. CI Pipeline	CI tool triggers build, runs unit tests, code scan, and quality checks.
3. Build Image	Build Docker image from Dockerfile and push to registry.
4. Container Registry	Store and manage container images securely.
5. Deploy	Apply Kubernetes manifests using kubectl to create/update resources.
6. Kubernetes Cluster	Scheduler places Pods on available Nodes and ensures desired state.
7. Service Exposure	Expose application using Service/Ingress for internal or external access.
8. Monitor & Scale	Monitor health, collect logs/metrics, auto-scale and self-heal applications.

Common Resources in Flow

- Deployment / StatefulSet
- Pod
- Service
- ConfigMap / Secret
- PersistentVolumeClaim
- Ingress
- NetworkPolicy
- ServiceAccount & RBAC

Interview Revision Cheat Sheet

1. Basics

- What is Kubernetes? Container orchestration platform.
- Why Kubernetes? Scaling, High Availability, Self-healing, Portability.
- K8s Architecture? Master (Control Plane) + Worker Nodes.
- What is a Pod? Smallest deployable unit (one or more containers).

2. Core Concepts

- Deployment – Manages ReplicaSet and Pods (stateless apps)
- Service – Stable networking to access Pods
- ConfigMap – Non-sensitive data
- Secret – Sensitive data (passwords, tokens)
- Namespace – Logical isolation within cluster

3. Storage

- PV – Cluster storage resource (provisioned by admin)
- PVC – Claim storage from PV
- StorageClass – Defines storage type & provisioner

4. Workloads

- StatefulSet – For stateful applications (DB, etc.)
- DaemonSet – Runs a Pod on all (or selected) nodes
- Job – Runs a task till completion
- CronJob – Runs Job on a schedule

5. Networking

- Pod IP – Ephemeral
- Service – Stable IP & DNS
- ClusterIP – Internal access
- NodePort – External via <NodeIP>:<Port>
- LoadBalancer – Cloud LB
- Ingress – HTTP/HTTPS routing

6. Scheduling

- Node Selector – Schedule on specific nodes
- Affinity – Advanced rules for placement
- Taints & Tolerations – Repel/allow Pods on nodes

7. Security

- RBAC – Role-based access control
- ServiceAccount – Identity for Pods
- NetworkPolicy – Control traffic to/from Pods

8. Observability

- kubectl logs – View logs
- kubectl describe – Details
- kubectl top – Resource usage
- Events – Troubleshooting
- Prometheus + Grafana – Monitoring stack

9. kubectl Essentials

- get – List resources
- describe – Details
- apply – Create/Update
- delete – Remove
- logs – Pod logs
- exec – Run command in Pod
- scale – Scale resources
- rollout – Rolling updates
- port-forward – Local access
- top – Resource usage

10. Common YAMLs

- Deployment.yaml
- Service.yaml
- ConfigMap.yaml
- Secret.yaml
- Ingress.yaml

11. Best Practices

- Use namespaces
- Use labels & selectors
- Keep containers lightweight
- Use ConfigMaps & Secrets
- Set resource requests/limits
- Enable liveness & readiness probes
- Use rolling updates
- Monitor & set alerts
- Follow least privilege (RBAC)

Remember

- concept before memorizing commands.
- Practice YAML manifest writing regularly.
- Use kubectl describe for troubleshooting.
- Observe, Monitor and Improve continuously.
- Security, Monitoring and Backup are critical.